Astrocyte: Experience Reuse and Collective Adaptation in Decentralized Agent Swarms

Authors: TO BE PROVIDED

19 October 2026 — research draft 0.1

Abstract

Reusable experience can influence how agents solve tasks and choose collaborators, while collaboration generates evidence that can revise both experience and future relationships. We propose Astrocyte as a framework for studying this coupled adaptation with fixed foundation-model parameters and constrained execution budgets. The proposed method combines versioned task and collaboration experience, condition-specific directed relationship scores, evidence-gated updates, and a shared decision rule that can be executed centrally or by individual members. A three-factor, eight-configuration protocol separates experience updates, relationship updates, and decision location. Its primary estimand is the interaction between the two updates under local decisions; secondary estimands measure how this interaction and the total update gain vary with decision location. Complete task sequences form paired replication units, and execution feedback is separated from sealed evaluation. We specify cost accounting, member perturbations, and a prospective system-iteration case. This manuscript presents a method and an unregistered experimental protocol. Current implementation evidence covers selected product components; the research policy, complete experimental controls, and formal measurements remain incomplete. No formal experimental results are reported, and no positive adaptation or decentralization effect is assumed.

Affiliations: TO BE PROVIDED. ORCID identifiers: TO BE PROVIDED.

Status. This is a complete first manuscript for review, based on the project positioning document v3 and experimental preregistration draft v0.1, both dated 19 October 2026. The protocol has not been formally registered. Model versions, tasks, budgets, sample sizes, and several analysis details require confirmation before registration. Nothing in this draft constitutes an empirical performance claim for the proposed method.

1 Introduction

An agent can preserve a useful procedure after completing a task. Reusing that procedure may change which task it attempts, which collaborator it requests, or which check it performs. Those choices change the evidence available for subsequent learning. A collaboration network can similarly favor partners whose recent contributions have worked under relevant conditions. These processes form a coupled feedback system: experience affects collaboration, and collaboration affects the production and revision of experience.

The scientific question is whether updating both processes yields a practically useful interaction over continuous tasks. A system might benefit from each update independently and still show little additional value from their combination. It might also exhibit a negative interaction when

an erroneous experience repeatedly directs work toward an unsuitable partner. Measuring only the full system against a static baseline cannot distinguish these possibilities.

We therefore propose a factorial study of experience updates, relationship updates, and the location of collaboration decisions. Foundation-model parameters remain fixed within the main study. The external experience state and condition-specific relationship records may change according to the assigned configuration. The word *decentralized* refers to members submitting their own task and partner decisions. Authorization, task claims, durable records, and budget enforcement may use shared services. Each Mission retains one authoritative execution backend.

The proposed contributions are an explicit representation connecting experience use to collaboration evidence; an evidence-gated relationship update with conditional scope and capability versions; and an experimental protocol that separates update interaction from deployment-dependent decision-location effects. The current contribution is the specification and its implementation audit. Establishing benefits requires the experiments described below.

The main study is distinct from the proposed system-iteration case. In the main study, the code and decision rule remain fixed across conditions. In the case, selected code or configuration changes are themselves prospective research artifacts. Mixing these settings would confound adaptation of external state with changes to the underlying agent system.

2 Related work and the scope of the research question

Strategy Genes studies compact, structured experience representations and selective incorporation of failure information with fixed model parameters (Wang et al. 2026, secs. 3–4 and Appendix A). This motivates controlling experience representation and its cost. Astrocyte's proposed factorial question concerns the joint effect of updating that experience and persistent collaboration relationships.

ShareMem separates transferable experience from user-bound preferences and studies two-stage consolidation and scoped retrieval (Hu et al. 2026, sec. 3 and Appendices A–B). It motivates recording coverage, provenance, and construction costs. Its results do not determine how continuous relationship updates interact with experience updates in the software-task setting proposed here.

EDV separates heterogeneous execution, third-party distillation, and executor consensus verification (Zhu et al. 2026, sec. 4 and Appendix G). Consensus remains a model judgment with possible shared bias. Astrocyte therefore proposes separate records for model agreement and independently checkable execution evidence.

MAAGL already uses historical trajectories for confidence estimation, collaborator selection, and experience learning in graph tasks (Qu et al. 2026, secs. IV-C–IV-D). Historical experience guiding partner selection is consequently an existing mechanism. The proposed distinction here is a controlled interaction study with condition-specific directed relationship updates over sequential software tasks. This is a research target, with novelty and effectiveness still requiring assessment. The cited author list follows the paper's HTML and PDF title pages; the abstract-page indexing discrepancy remains documented in the source audit.

Market-based post-training jointly trains action and bidding policies and evaluates population changes (Xu et al. 2026, sec. 2 and 3.4). Astrocyte proposes external-state adaptation with fixed model parameters and separately manipulates decision location. Member perturbations are a relevant evaluation dimension, with their effects to be measured under the present protocol.

Darwin Gödel Machine maintains an archive of code-modified agents and evaluates candidate changes (Zhang et al. 2025, secs. 3–4 and Appendix F). It motivates the prospective iteration case and the need to protect evaluation from candidate-controlled instrumentation. EvoMap's official Swarm documentation describes shared memory and history-sensitive collaboration (EvoMap, n.d., "Shared Memory" and "Intelligent Orchestration"). That documentation provides design context; it supplies no causal estimate for Astrocyte.

The reviewed sources establish substantial prior work on experience reuse and adaptive coordination. This draft makes no claim to an exhaustive literature review or priority for combining memory with collaborator selection. The intended evidence would concern the specified update interaction and its limits.

3 Setting and state

Let a task sequence contain T scheduled tasks. A member has an identity, a known capability version, a current availability state, and an authorization scope. A decision at time t uses information available before submission. Foundation-model versions and inference settings are recorded; no parameter training occurs in the main experiment.

The adaptive state consists of an experience collection E_t and relationship records R_t. Immediate state Z_t includes task progress, outstanding claims, current load, availability, and permissions. Separating these states is essential: disabling learning must preserve operational permission checks and changes in availability.

Tasks have versioned condition features x_t. Proposed software-task features include dependency structure, file overlap, available tools, modification type, environment compatibility, and verification requirements. A feature record includes its extraction time, source, missingness, and `context_schema_version`. Only information visible before the decision is eligible. Features derived from final patches, failures, or sealed evaluation cannot enter that decision's matching rule.

The same feature object supports experience applicability and the grouping rule that produces `scope_id` for relationships. Permission and environment compatibility filter candidates. Structural and semantic similarity rank eligible evidence. The concrete extractor, normalization, grouping boundaries, and missing-value rules require development validation and registration. This manuscript specifies their role without claiming that the current product implements them.

4 Proposed method

4.1 Versioned experience and use records

An experience unit records the task conditions, suggested actions, collaboration arrangements, participant capability versions, source events, verification evidence, failure conditions, cost, use history, and version lineage. Task experience describes problem-solving or tool-use procedures. Collaboration experience describes role assignments, handoff requirements, and checks on contributions.

A retrieval hit establishes availability of an experience. Actual adoption requires a use record linking its exact version to a submitted decision and subsequent execution. This distinction permits measurement of exposure, adoption, and downstream outcomes without counting all retrieved entries as used.

The research draft distinguishes unverified, single-task-verified, and multi-condition-verified experience. Unverified entries enter an isolated verification queue. Ordinary decisions retrieve only entries meeting the applicable evidence requirement **and** current authorization. In the development defaults, one independent successful task supports single-task verification; at least three independent tasks covering two predefined conditions support multi-condition verification. These are draft operating rules, with no claim that three tests establish general reliability.

An applicable counterexample suspends ordinary recommendation and prompts revision and revalidation. Repeated checks on one task do not create independent task coverage. Evidence records distinguish automated tests, checkable tool results, blinded human evaluation, and model judgments.

Evidence strength and permission to use an entry are separate concepts. Existing human approval must remain effective. A future evidence-level implementation cannot automatically grant the current product's approval or cross-project sharing permission. This separation requires an explicit product decision before execution.

4.2 Conditional directed relationships

A relationship key includes requester i, collaborator j, their known capability versions, collaboration type, and condition scope c. Its score uses discounted verified successes and failures:

$$s_{ijc}^{t+1} = \lambda^{\Delta t} s_{ijc}^{t} + q_t r_t, \qquad f_{ijc}^{t+1} = \lambda^{\Delta t} f_{ijc}^{t} + q_t (1 - r_t),$$

$$w_{ijc}^{t+1} = \frac{\alpha_0 + s_{ijc}^{t+1}}{\alpha_0 + \beta_0 + s_{ijc}^{t+1} + f_{ijc}^{t+1}}.$$

Here $r_t \in \{0, 1\}$ is the result of a check attributable to the specific handoff or collaboration, and $q_t \in [0, 1]$ weights admissible evidence. The development primary rule uses $q_t = 1$ for evidence satisfying a predeclared check and $q_t = 0$ for model opinion alone, executor self-report, ordinary process exit, or unresolved attribution. Human evidence must follow a fixed rubric, hide configuration, and identify the target event.

Each evidence event contributes once. Multiple tests or duplicated transport events for one relationship outcome within a task must not create multiple complete counts. Task-level success is insufficient to credit every relationship in a collaboration chain; unresolved events remain pending attribution. Scores describe discounted evidence and require prospective calibration. Their prior-like algebra does not establish calibrated Bayesian posterior probabilities, particularly with forgetting, selected partners, and correlated tasks.

The development defaults use $\alpha_0 = \beta_0 = 1$ and a half-life of 20 task rounds, giving $\lambda = 2^{-1/20}$. The operational round clock, the time at which decay becomes visible, and delayed-evidence ordering must be specified before registration. When updates are enabled, a zero-weight event can still coincide with elapsed-time decay; it supplies no new success or failure. When relationship updates are disabled, counts, weights, and decay remain frozen at the initial snapshot.

Unavailable members leave the eligible set immediately. An announced capability-version change creates the corresponding relationship key, with any historical transfer governed by the registered

condition rule. An unannounced capability change initially affects the existing record through subsequent observable outcomes. Neither case changes permissions automatically.

4.3 Common decision rule

For eligible candidates C, the proposed `policy_score` combines normalized task fit, experience support, relationship score, estimated cost, and current load. Development defaults add the first three and subtract the last two, with coefficients one. Accurate feature definitions, aggregation, and cost prediction remain registration fields.

For candidate j with score U_j, choose according to

$$p(j) = (1-\epsilon)\frac{\exp(U_j/\tau)}{\sum_{k\in C}\exp(U_k/\tau)} + \frac{\epsilon}{|C|}.$$

The draft defaults are $\epsilon = 0.10$ and $\tau = 1$. Candidate ordering, score components, probabilities, random draws, selected experience versions, observation time, state version, submission time, and claim outcome must be observable. An empty candidate set produces an unassigned decision under the common waiting rule. Exploration operates inside capability and authorization constraints.

Central and local decisions call the same implementation. For a decision assigned to member i, the central service receives that member's information view, candidate set, retrieval allowance, and decision budget. Both use the same claim and conflict-handling service and the same total decision-concurrency allowance. The central service submits choices for members; local members submit their own choices.

4.4 Feedback cycle and failure handling

After an executed task, the allowed feedback suite produces evidence. The experience process proposes revisions and verifies candidates within its assigned budget. Admissible, attributable outcomes update the corresponding relationships. Later decisions read only the state allowed by their configuration. The intended cycle is specified below; it is not an executed algorithm trace.

```
For each scheduled task in an isolated configuration:
    obtain current authorized state and the permitted adaptive-state versions
    construct the pre-decision conditions and eligible candidates
    retrieve eligible experience and compute the common decision probabilities
    submit the selected task or partner decision through the common claim service
    execute within the registered task and sequence budgets
    record allowed feedback, attribution, usage, cost, and unresolved outcomes
    if experience updates are enabled: verify and revise eligible experience
    if relationship updates are enabled: apply admissible evidence and declared decay
    submit the final artifact to the isolated evaluator
```

Unavailable shared services prevent affected new decisions and produce an explicit failure or waiting record. Recovery must follow the registered policy, including any explicitly selected snapshot mode and its version. No unobserved retry, substituted state, or unknown-delivery resend can be treated as successful execution. Duplicate evidence handling reuses event identity; it does not create a separate scheduling authority.

5 Experimental protocol draft

5.1 Eight paired configurations

Every replication contains the following eight configurations in randomized execution order. All begin from the same nonempty experience and relationship snapshots, copied into isolated condition state, and process the same scheduled task sequence.

Configuration	Decision location	Experience updates	Relationship updates
L00	Local	Off	Off
L10	Local	On	Off
L01	Local	Off	On
L11	Local	On	On
C00	Central	Off	Off
C10	Central	On	Off
C01	Central	Off	On
C11	Central	On	On

All conditions can retrieve initial experience. Experience-off freezes content, verification state, historical use statistics, and ranking terms derived from those statistics. Relevance to a current task may be recomputed from the frozen representation. Relationship-off freezes counts, weights, and time decay. Immediate task state, permissions, and availability continue to change in every condition. Logs remain writable, but off-condition decisions cannot learn from those logs through caches, indexes, summaries, or shared services.

Model lists, prompts, source collections, feature extraction, task order within each paired block, concurrency quotas, and task/sequence budget rules are shared. New literature or other knowledge cannot enter only selected conditions. Initialization and execution order are recorded together with service versions and calendar time to expose drift and shared-service interference.

5.2 Decision location and identification

A synchronous development replay supplies equal information snapshots, event order, candidate identities, and random draws to both implementations. Matching selections test implementation consistency. With equal state and random draws, the common rule implies equal decisions; replay supplies no evidence for a positive location effect.

The deployed comparison records differences in state age, request waiting, submission ordering, claim conflicts, and communication. A nonzero location effect would describe those specified deployment conditions. It cannot be attributed to abstract decentralization while central caching, extra information, different scoring, or unequal compute also vary. Shared storage, process placement, cache policy, communication path, and scheduling must therefore be declared. Whether the deployment introduces a meaningful location intervention remains a readiness question.

5.3 Tasks, feedback, and sealed evaluation

Each task has a `feedback_suite` and an `evaluation_suite`. Feedback is available to execution and the permitted updates. Evaluation answers, item-level outcomes, and scores stay inaccessible to those processes until the entire task sequence ends. An independent evaluation process receives

final artifacts and retains its results separately. Ordinary completion messages and retrieval indexes must not return sealed scores indirectly.

Task and suite selection are unfinished. The registration record must identify exact versions, permissions, overlap between suites, and the public-project scope. Isolating results does not guarantee independence when the feedback suite reveals the same assertions or target answers; overlap and adaptation to public tests require explicit review.

The primary task outcome is binary satisfaction of the fixed evaluation suite. Errors, timeouts, budget exhaustion, and unfinished tasks count as failures. After a sequence exhausts its budget, all remaining scheduled tasks remain in the denominator. Damaged task data or missing evaluation programs follow predeclared, configuration-symmetric exclusion rules. Algorithmic failures, unhelpful experience, invalid tools, waiting, and retries remain outcomes and costs. A retry never erases its original execution record.

5.4 Budgets and accounting

Accounting covers model calls, tools, experience extraction, retrieval, verification, coordination, retries, and initialization. The unverified-experience queue has a draft ceiling of 5% of its configuration's total budget; its work is charged to that configuration. This ceiling and all other budgets require confirmation.

Model usage distinguishes input, cached input, output, and reasoning tokens when observable, by exact model version and billing period. Payment and resource usage are separate: a subscription-covered request can have no incremental payment while consuming resources. Unknown usage remains unknown and cannot be recorded as zero. Missing budget-relevant evidence blocks a formal budget-matched claim.

Communication records message count, bytes, model-context tokens, waiting time, and elapsed time. Snapshot production is reported as an actual shared preparation expense and as an explicitly amortized per-run amount. The allocation rule must be registered; generating a snapshot once does not create eight independent preparation events. Conversely, reusing a public snapshot does not justify hiding its production cost in end-to-end comparisons. Paired executions remain isolated even when they start from the same snapshot.

Let the main study use r replications of T tasks, perturbations use r_p replications of T_p tasks in three configurations and four scenarios, and the case use r_s replications of three processes over G rounds and T_s evaluations per round. The draft task-accounting identity is

$$N_{\text{tasks}} = 8rT + 12r_pT_p + 3r_sGT_s + N_{\text{pilot}} + N_{\text{initial}}.$$

Case candidate generation and development verification are charged additionally. The input's $r = 5, T = 10$ example gives 40 sequences and 400 task executions; its assumed 30 calls per task gives approximately 12,000 calls before preparation and development work. These are budget illustrations, with no observed call rate or approved formal sample size.

6 Estimands and analysis

6.1 Primary and secondary contrasts

For replication k, let

$$Q^P_{ab,k} = \frac{1}{T_k} \sum_{u=1}^{T_k} Y^P_{ab,k,u}, \qquad P \in \{\text{local}, \text{central}\}, \quad a, b \in \{0, 1\}.$$

T_k is the common post-exclusion scheduled denominator across the eight configurations. We compute each replication's contrasts first, then average contrasts equally across replications. The planned position-specific interaction is

$$I^P_{\text{feedback}} = Q^P_{11} - Q^P_{10} - Q^P_{01} + Q^P_{00}.$$

The primary target is $I^{\text{local}}_{\text{feedback}}$. The draft practical target $\delta_{\text{feedback}} = 0.03$ means three additional successes per hundred scheduled tasks beyond the additive expectation from the individual updates. It does not mean that L11's total gain over L00 is three points.

Decision-location modification of the interaction is

$$I_{\text{three}} = I^{\text{local}}_{\text{feedback}} - I^{\text{central}}_{\text{feedback}}.$$

Decision-location modification of the total joint-update gain is

$$I_{\text{position}} = (Q^{\text{local}}_{11} - Q^{\text{local}}_{00}) - (Q^{\text{central}}_{11} - Q^{\text{central}}_{00}).$$

These are distinct estimands. In each position,

$$Q^P_{11} - Q^P_{00} = I^P_{\text{feedback}} + (Q^P_{10} - Q^P_{00}) + (Q^P_{01} - Q^P_{00}).$$

Consequently, the location difference in total gain also contains location differences in the two individual update gains. A large L11–L00 improvement alone does not establish interaction. We additionally report L11–L10, L11–L01, and L11–C11 together with costs. Positive interaction and acceptable operational value are separate requirements.

6.2 Replication and uncertainty

The independent replication unit is a complete paired eight-configuration block with independently executed requests and reinitialized state. Tasks within a sequence are dependent because earlier outcomes change later experience and relationships. Repeated tests, multiple analyses of a run, model seeds, and eight condition labels do not multiply the number of independent replications.

The main analysis resamples complete replication blocks, preserving all eight conditions and their sequence/initialization pairing, and estimates two-sided 95% intervals. The resampling count, exact interval construction, and validity checks require registration. Few independent blocks can yield

unstable interval tails. More tasks per sequence cannot substitute directly for more independent blocks.

If the same fixed task set and development snapshot are reused across replications, inference is conditional on that set and snapshot. Population-level claims about unseen tasks or initialization histories require corresponding independent sampling and analysis. Independent reinitialization means restoring a separate state copy; it does not by itself make the initial history independently sampled.

Under the draft decision rule, an interval entirely above δ_{feedback} supports the practical primary target; an interval entirely below it does not reach that target; an interval crossing it is inconclusive. A two-sided 95% lower bound corresponds nominally to a one-sided 2.5% boundary in a calibrated continuous setting. Power planning must reflect that choice.

The three position-related secondary comparisons are I_{three}, I_{position}, and $Q_{11}^{\text{local}} - Q_{11}^{\text{central}}$. The draft calls for Holm adjustment of their difference tests. Once valid marginal tests are specified, sorted p values use $p_{(j)}^{\text{adj}} = \min\{1, \max_{i \le j}(3-i+1)p_{(i)}\}$. Null calibration and the marginal testing procedure remain unresolved; a confidence interval alone does not supply those tests. No adjusted significance claim is available from this draft.

The proposed position-equivalence range is $[-0.02, 0.02]$. We will report whether an entire relevant interval lies inside that range. Failure to detect a difference does not establish equivalence. The equivalence testing procedure and its multiplicity family require confirmation; the draft does not silently turn difference-test adjustment into equivalence evidence.

Adoption comparisons, perturbations, and process analyses must have explicit reporting levels and any confirmatory families declared before formal data collection. Transparent reporting and effect magnitude are consistent with the ASA communication cited here (American Statistical Association 2016); that press release supplies no technical justification for the particular resampling or multiplicity methods.

6.3 Sample-size planning

Formal replication counts depend on pilot estimates of the sequence-level interaction variance, task dependence, the minimum useful effect, and budget. The supplied planning example considers a true interaction of 0.06 against a target of 0.03 at 80% power. Under an approximate independent-block normal model, a planning calculation has the form

$$r \approx \left(\frac{(z_{0.975} + z_{0.80})\sigma_I}{0.06 - 0.03} \right)^2,$$

where σ_I must come from independent pilot block contrasts. This is an approximation for planning and requires simulation checks with development sequences. It supplies no numerical sample size without a variance estimate. Replication counts, maximum runs, stopping rules, and the final analysis procedure remain open. Formal results must not determine these choices retrospectively.

7 Adaptation, mechanisms, and prospective iteration case

7.1 Member perturbations

The protocol proposes member exit, entry, announced capability change, and unannounced capability change, comparing L11, L00, and C11. Perturbation times, proportions, identities, and capabilities remain scenario-registration fields. Matched configurations begin from corresponding states and schedules. An exit updates availability immediately; its effect on ongoing work requires real lifecycle evidence.

Measures include task transfer, incorrect delegation, detection delay, relationship adjustment, recovery time, quality loss, and cost. Selection records reveal whether a new member receives opportunities and whether reused experience supports subsequent work. A stopped Mission with no Worker provides no evidence about adaptation to member exit.

7.2 Structural distance and negative interactions

Task distance is computed before execution relative to the initial historical tasks. The draft uses normalized clipped numerical differences, categorical disagreement, and set differences based on intersection and union, averaged over observed features. Insufficiently observed tasks form an unknown group. The exact scale, nearest-history rule, and near/middle/far boundaries must be fixed using development data.

Distance groups share the same tasks across conditions. Group-level L11–L00 gains are reported with coverage and difficulty indicators. Distance is observed rather than randomized, so a gain-distance pattern does not independently establish a causal transfer law.

Process records support examination of harmful experience reuse, concentration of assignments, stale capability versions, delayed updates, and learning overhead. These are candidate explanations. Attribution requires separately labeled interventions, such as disabling a particular experience or changing update visibility in a paired rerun. Replaying decisions cannot reproduce unobserved model or tool outcomes in a different environment. Observational logs alone cannot establish the cause of a negative interaction.

7.3 Swarm A/B/C application case

The proposed case assigns hypothesis formation to A, candidate modification to B, and development verification and independent evaluation to C. Each candidate records a prediction, applicability conditions, code/configuration version, development checks, adoption decision, cost, and lineage. These are functional responsibilities; this manuscript does not claim that autonomous A/B/C iteration has occurred.

Three processes compare persistent cross-round experience, reset cross-round experience, and an unchanged system under matched budgets. Development feedback supports candidate selection. Sealed evaluation is controlled by the evaluator; its results must not influence later candidate generation during the evaluated sequence of rounds. Repeated access to the same final holdout would compromise that separation.

Round-level quality, transfer to new tasks, adopted-improvement cost, and cumulative performance will be reported when measured. A hypothesis-acceptance rate describes the research process and

cannot substitute for independent performance. All candidates, failures, and costs must remain in the case record. This first draft contains no invented round table.

8 Current implementation and available evidence

The inspected product candidate is `7ce3a5efe312f4cdac3491293e002f5559f076b0`; its business-code source is `982333b871520e606c74d553e448c5049f81abde`. The research writing branch began from a separate repository default and does not imply an updated product checkout. The following evidence levels are kept distinct.

Level	Current scope	Implication for this manuscript
Proposed method	Conditional experience, relationship updates, common probabilistic policy, eight configurations	Specified here; complete execution remains unverified
Inspected implementation	Mission/WorkItem and lease logic; fixed candidate ranking; human-governed experience; accepted-source linkage	Reusable product components with narrower semantics
Independent engineering report	Selected manually created DAG, permission, reconnect, lifecycle-without-Worker, and loading checks	Component evidence reported by a separate agent session
Formal research results	Eight-configuration study, member adaptation, autonomous iteration	Not collected

In `internal/swarm/domain/policy.go`, `swarm.fixed.v1` filters by approval, budget, capabilities, availability of dependencies, and write-path conflicts, then orders by matching inputs and identifier. It does not establish the proposed experience-and-relationship softmax policy. The inspected `internal/attention/domain/experience.go` and application service distinguish quarantine and approved entries with human governance. Accepted-artifact linkage exists in `internal/swarm/app/experience.go`. These findings support an implementation plan; they do not demonstrate realized experience-driven collective adaptation.

The independent report is pinned separately (*Astrocyte Dynamic DAG Independent Engineering Review* 2026). Its accepted scenarios use manually created work items and do not validate model-generated task planning or Worker exit. This research worker read that report and inspected the stated source; it did not rerun the product tests or independently query their event database.

Restricted native execution remains blocked by the requirement to disable native file and command tools. The pending S3 decisions concern call-budget accounting and the evidence required for tools and external destinations; S3 here names the project's Swarm execution stage. They must be resolved for the affected research execution path. Unknown delivery cannot be retried as though unsent. OpenViking's positive remote path, actual work completion/adoption, and subsequent experience reuse remain unverified.

Full CI has not passed. The reported automated runs were ultimately cancelled, and their Ubuntu full-repository Go test jobs had already failed before cancellation. Some earlier fixture and migration inconsistencies were identified; failure attribution was incomplete. Limited component acceptance cannot establish whole-product correctness.

The separately governed V4 study is outside this protocol. Its original external evidence root is unavailable according to the current project handoff. Historical aggregates without their inspectable raw evidence are excluded from the present research results. No V4 configuration, lock, database, score, or automation was changed for this draft.

9 Results status and interpretation limits

No main-study, perturbation, or iteration-case results have been collected for this manuscript. There is no estimated Q, update interaction, interval, significance value, equivalence conclusion, cost improvement, or measured adaptation curve to report. Implementation inspection and literature reading establish readiness information only.

When data become available, reporting must include all eight condition outcomes, paired contrasts, uncertainty, failure denominators, cost components, and the actual sampling population. A beneficial total update gain with negligible interaction would support useful additive effects. A negative interaction would motivate controlled mechanism investigation. A near-zero location contrast with sufficiently precise intervals could describe similar behavior under the tested deployment; a wide interval would remain inconclusive. None of these outcomes is selected in advance.

10 Limitations, reproducibility, and research governance

The method is incomplete as an executable research system. Proposed condition features may omit relevant task differences, and partner selection changes which relationships are observed. Calibration among selected partners cannot automatically generalize to unselected pairs. Fixed software tasks and one initial history limit external validity. Provider-side randomness and model-version changes can weaken paired comparisons even when requested seeds match.

The evidence-quality defaults do not guarantee broad applicability. Feedback tests may be exploitable or overlap with final evaluation; human ratings may vary; model agreement can share errors. Authorization and privacy remain constraints on available evidence, with no permission inferred from the existence of an experience store. Initial snapshots and the formal task corpus require lawful, explicit provenance.

Before registration, the reproducibility package must identify the exact code and policy versions, all model/service versions, prompts and decoding settings, task sequences and suite access, initial-state sources, per-condition isolation, hardware and communication placement, resource limits, costs, randomization, exclusions, replication counts, and analysis programs. Existing project services and records should be reused. This requirement does not authorize a new scheduler or proof-of-completion system.

Formal execution depends on legitimate native-controller configuration, real persistent state and HTTP services, and the actual session files produced by the controller. Each claimed event must be locatable in the real event database. The writing work reported here sends no new product-native model input and runs no model experiment. Development writing tools are separate from

permissions for the proposed experimental agents.

The supplied protocol remains a draft. Authors, affiliations, ORCID identifiers, final thresholds, budgets, tasks, replication counts, and a registration identifier are unfilled. Public submission and formal registration require subsequent decisions. The manuscript is intended for scientific and implementation review in its present evidence state.

11 Conclusion

Astrocyte specifies a study of coupled external experience and collaboration-relationship updates under fixed model parameters. The eight-configuration design distinguishes update interaction, decision-location modification of interaction, and decision-location modification of total gain. Its evidential requirements include paired complete sequences, isolated adaptive states, protected evaluation, and comprehensive accounting. Current product evidence supports selected components; the proposed research effects remain to be measured.

12 References

American Statistical Association. 2016. *American Statistical Association Releases Statement on Statistical Significance and p-Values*. https://www.amstat.org/asa/files/pdfs/p-valuestatement.pdf.

Astrocyte Dynamic DAG Independent Engineering Review. 2026. https://github.com/songconmaisaix31-design/Astrocyte/blob/7f781deea143c74bed5303f90d8130c82bb54bca/docs/reviews/dynamic-dag-20261019.md.

EvoMap. n.d. *Swarm Intelligence*. Accessed October 19, 2026. https://evomap.ai/wiki/10-swarm.

Hu, Jinming, Haodong Zhao, Qi Jia, et al. 2026. *Learning from Others, Acting for You: Cross-User Memory Sharing for LLM Agents*. https://arxiv.org/html/2609.32511v1.

Qu, Liang, Jianxin Li, Hongzhi Yin, and Hua Wang. 2026. *Multi-Agent Agentic Graph Learning via Structural Signatures*. https://arxiv.org/html/2609.09565v1.

Wang, Junjie, Yiming Ren, and Haoyang Zhang. 2026. *From Procedural Skills to Strategy Genes: Towards Experience-Driven Test-Time Evolution*. https://arxiv.org/html/2604.15097v4.

Xu, Guowei, Zhenting Qi, Amani Kiruga, Himabindu Lakkaraju, and Yilun Du. 2026. *Learning Self-Orchestration in Decentralized Agent Swarms*. https://arxiv.org/html/2610.22322v1.

Zhang, Jenny, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. 2025. *Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents*. https://arxiv.org/html/2505.22954v1.

Zhu, Shiding, Yudi Qi, Yajie Wang, et al. 2026. *Escaping the Self-Confirmation Trap: An Execute-Distill-Verify Paradigm for Agentic Experience Learning*. https://arxiv.org/html/2606.24428v1.